

Desenvolvimento do Jogo “Mega Man x Mario” Utilizando Assembly RISC-V - Grupo A7

Adria Carolina de Melo Dias¹,

Ana Luiza Campos Coelho¹,

Arthur Vilaça Serodio¹,

Gabrielly Batista de Freitas¹,

Gustavo Pires dos Santos¹,

João Francisco de Sousa Torres¹

¹Dep. Ciência da Computação – Universidade de Brasília (UnB), 15 de julho de 2026

CIC0099 – Organização e Arquitetura de Computadores

251032290@aluno.unb.br, 251035926@aluno.unb.br, 251039335@aluno.unb.br, 251004400@aluno.unb.br,
251004349@aluno.unb.br, 251037072@aluno.unb.br

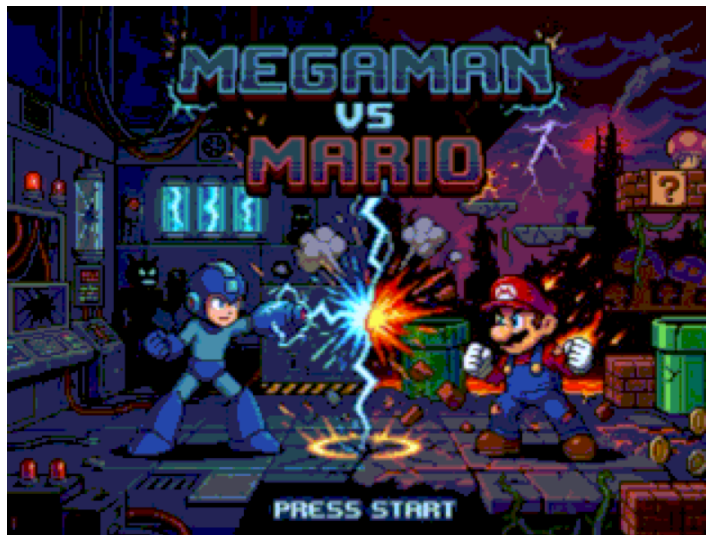


Figura 1: Tela inicial do jogo

Resumo

Neste trabalho é apresentado o desenvolvimento de um jogo inspirado em Mega Man 2, implementado integralmente em linguagem Assembly RISC-V utilizando o ambiente FPGRARS, conforme os requisitos propostos pela disciplina Organização e Arquitetura de Computadores da Universidade de Brasília. Diferentemente de uma simples reprodução do jogo original, o projeto propõe misturar os universos Mega Man e Super Mario Bros., incorporando elementos visuais e mecânicos característicos de ambas as franquias. O jogo desenvolvido possui duas fases com ambientações distintas, sistema de movimentação com rolagem horizontal do cenário, combate baseado em projéteis e combate corpo a corpo, habilidades alternáveis,

itens coletáveis, efeitos sonoros e chefes exclusivos para cada fase. Os resultados obtidos demonstram que é possível desenvolver uma aplicação gráfica relativamente complexa utilizando exclusivamente Assembly RISC-V, explorando conceitos fundamentais de arquitetura de computadores.

Palavras-chave: Assembly RISC-V, FPGRARS, Desenvolvimento de Jogos, Mega Man, Mario, Organização e Arquitetura de Computadores.

1. Introdução

O desenvolvimento de jogos digitais em linguagem Assembly representa um desafio devido ao baixo nível de abstração da linguagem e à necessidade de gerenciar diretamente recursos como memória,

entrada de dados, renderização gráfica e controle do fluxo de execução.

Neste trabalho foi desenvolvido um jogo inspirado em *Mega Man 2*, utilizando a arquitetura RISC-V e o ambiente FPGRARS como plataforma de desenvolvimento e testes. O projeto foi feito para execução em uma placa FPGA, aproximando o desenvolvimento realizado em simulador da execução em hardware real. Como diferencial, o projeto propõe um crossover entre os universos Mega Man e Super Mario Bros., apresentando duas fases com ambientações distintas, inimigos e chefes inspirados em ambas as franquias. Ao longo da partida, o jogador deve atravessar os cenários enfrentando diferentes inimigos, coletando itens e utilizando habilidades para derrotar os chefes de cada fase.

O principal objetivo do projeto foi desenvolver uma aplicação completa em Assembly RISC-V capaz de integrar sistemas de movimentação, combate, animação, colisão, interface gráfica, áudio e inteligência artificial simples. Nas seções seguintes são apresentados os conceitos utilizados, a metodologia adotada durante o desenvolvimento e os resultados obtidos com a implementação do jogo.

2. Fundamentação Teórica e Trabalhos Relacionados

O desenvolvimento do projeto baseou-se nos conceitos apresentados ao longo da disciplina de Organização e Arquitetura de Computadores, especialmente aqueles relacionados à programação em linguagem Assembly para a arquitetura RISC-V [1], [2].

Diferentemente de linguagens de programação de alto nível, nas quais diversas operações são abstraídas pelo compilador e pelas bibliotecas disponíveis, a programação em Assembly exige que todas as instruções sejam implementadas explicitamente pelo desenvolvedor. Operações como movimentação de dados, gerenciamento de memória, renderização gráfica e controle do fluxo de execução precisam ser realizadas diretamente por meio das instruções da arquitetura RISC-V. Essa característica torna o desenvolvimento mais complexo, porém proporciona uma compreensão mais aprofundada do funcionamento interno do processador e da organização de sistemas computacionais [1], [2].

O jogo foi desenvolvido utilizando o ambiente

FPGRARS, uma extensão do simulador RARS voltada ao desenvolvimento de aplicações gráficas e multimídia em Assembly RISC-V. O ambiente disponibiliza periféricos simulados, como teclado, bitmap display e sistema de áudio, possibilitando a implementação de aplicações interativas e jogos completos sem a necessidade de hardware dedicado [3], [4].

Além da utilização do simulador durante o desenvolvimento, o projeto foi planejado para execução em uma placa FPGA utilizada na disciplina. Dessa forma, o FPGRARS foi empregado como ferramenta de desenvolvimento e validação das funcionalidades implementadas antes da execução em hardware, permitindo identificar e corrigir problemas de forma mais eficiente durante o processo de implementação [4], [6].

Para a construção dos elementos gráficos, foram utilizados sprites inspirados nas franquias Mega Man e Super Mario Bros. [8], [9], [10]. Durante o desenvolvimento também foram analisados projetos produzidos em semestres anteriores da disciplina, o Livro de OAC e outros materiais de apoio disponibilizados durante a disciplina [6], [7].

Além disso, foram utilizados materiais e exemplos disponibilizados no repositório LAMAR, que reúne ferramentas e projetos desenvolvidos em Assembly RISC-V [5].

3. Metodologia

O desenvolvimento do jogo foi realizado de forma incremental, implementando inicialmente as mecânicas fundamentais e, posteriormente, adicionando novas funcionalidades até atender aos requisitos propostos pela disciplina.

Para facilitar o desenvolvimento em equipe, o código foi organizado em módulos responsáveis por diferentes componentes do jogo, como renderização gráfica, movimentação, combate, gerenciamento das fases, interface gráfica e sistema de áudio.

3.1 Estrutura geral

A estrutura do projeto foi dividida em três diretórios principais: docs, src e tests. O diretório docs contém os arquivos de documentação produzidos durante o desenvolvimento. Já o diretório src reúne todos os arquivos necessários para execução do jogo, incluindo o código Assembly, bibliotecas auxiliares, simulador FPGRARS e recursos gráficos utilizados durante a

renderização. Por fim, o diretório tests foi reservado para futuras implementações de testes.

O código principal encontra-se no arquivo jogoOAC.asm, responsável pela inicialização do jogo e execução do game loop. Para reduzir a repetição de código, foi utilizada a biblioteca MACROsv24.s, que reúne macros frequentemente utilizadas durante o desenvolvimento. Além disso, o arquivo SYSTEMv24.s disponibiliza rotinas relacionadas às chamadas de sistema (ecalls) e ao tratamento de exceções.

Os recursos gráficos foram organizados dentro do diretório script, que contém tanto os arquivos de imagem utilizados durante o desenvolvimento quanto suas versões convertidas para o formato compatível com o FPGRARS.

```

megaman-OAC-grupo7/
├── docs/
│   └── situacao-atual.md      # Documento de planejamento (desatualizado)
├── src/
│   ├── asm/                  # Código-fonte principal
│   │   ├── jogoOAC.asm       # Jogo principal (~450 linhas)
│   │   ├── MACROsv24.s       # Biblioteca de macros (v2.4)
│   │   ├── SYSTEMv24.s       # Sistema de ecalls e tratamento de exceções
│   │   ├── main.s            # Placeholder vazio
│   │   └── fpgrars-x86_64-pc-windows-msvc--unb.exe # Simulador RARS customiz
│   └── script/
│       ├── script.exe         # Utilitário de conversão de imagens
│       ├── bmp2/              # Conversores BMP → .data
│       ├── imagens_para_conversao/ # Assets fonte (BMP/PNG)
│       └── imagens_convertidas/  # Assets convertidos (.data)
├── verilog/
│   └── top.v                  # Módulo Verilog (vazio - a implementar)
└── tests/                    # (vazio)

```

Figura 2: Estrutura geral do projeto

3.2 Renderização

A renderização do jogo foi implementada utilizando o Bitmap Display disponibilizado pelo FPGRARS. Os elementos gráficos são desenhados em um buffer oculto e, ao final de cada iteração do laço principal, ocorre a troca entre os buffers de exibição.

O cenário é composto por mapas previamente convertidos para o formato utilizado pelo simulador, sendo exibido de forma dinâmica conforme a posição do personagem. O deslocamento horizontal da câmera acompanha a movimentação do jogador, permitindo percorrer fases maiores do que a área visível na tela.

Além dos elementos do cenário, também são renderizados o personagem, os projéteis, os inimigos, os itens coletáveis e a interface gráfica do jogo. A HUD é desenhada pela rotina DESENHA_HUD, sendo responsável por apresentar ao jogador informações como quantidade de vidas, energia disponível para utilização das habilidades e arma atualmente equipada.



Figura 3: Cenário da primeira fase, exibindo a HUD, um inimigo e um item coletável

3.3 Mecânica do jogo

As mecânicas do jogo foram desenvolvidas com o objetivo de reproduzir as principais características da série Mega Man, adaptando-as ao contexto do projeto e incorporando elementos do universo Super Mario Bros.. Para isso, foram implementados sistemas de movimentação, combate, interação com o cenário e progressão entre fases, proporcionando uma experiência de jogo dinâmica e compatível com os requisitos estabelecidos para o projeto.

A movimentação do personagem permite deslocamento horizontal, salto e interação com plataformas e obstáculos presentes no cenário. O controle do salto foi implementado por meio de uma simulação simplificada da gravidade, em que a posição vertical do personagem é atualizada continuamente até seu retorno ao solo. Durante esse processo, o sistema de colisões verifica constantemente a interação entre o personagem e o mapa, impedindo a passagem por regiões sólidas e garantindo uma movimentação consistente ao longo das fases.

O sistema de combate foi baseado no ataque característico do personagem Mega Man, utilizando projéteis disparados na direção em que o jogador está orientado, sendo também possível alterar a forma de combate para corpo a corpo, possibilitando que o Mega Man dê socos nos inimigos.

Durante a execução do jogo, são realizadas verificações contínuas de colisões. Essas verificações controlam tanto a eliminação dos inimigos quanto o sistema de vidas do jogador, garantindo o correto funcionamento das mecânicas.

Os inimigos foram implementados com comportamentos próprios, oferecendo diferentes desafios ao jogador ao longo da partida. Ao final de

cada fase ocorre o confronto contra um chefe exclusivo: Bowser na primeira fase e Mario na segunda.

Além do combate, o jogo conta com itens coletáveis distribuídos ao longo dos cenários, responsáveis pela recuperação da energia utilizada pelas habilidades e para curar vidas perdidas.

A progressão do jogo ocorre por meio de duas fases com ambientações distintas, conectadas por um portal localizado ao final da primeira fase. Após atravessá-lo, o jogador é transportado para um novo cenário, mantendo o estado da partida e dando continuidade à aventura até o confronto final e a exibição da tela de vitória (ou de derrota, caso o jogador perca todas as vidas).



Figura 4: Portal de mudança de fase

3.3 Inimigos e Chefões

Os inimigos foram desenvolvidos para oferecer diferentes desafios ao longo das fases, incentivando o jogador a utilizar tanto o combate à distância quanto o combate corpo a corpo.

Cada tipo de inimigo possui um padrão de comportamento específico, tornando a progressão do jogo mais dinâmica e exigindo diferentes estratégias durante os confrontos.

Na primeira fase, o jogador encontra plantas carnívoras, posicionadas em locais fixos do cenário. Também nessa fase aparece o Bowser Esqueleto, inimigo que realiza deslocamento horizontal pelo cenário, representando uma ameaça constante durante a exploração. Tocar em qualquer um dos dois faz o jogador perder uma vida.



Figura 5: Inimigos da primeira fase

Ao final da primeira fase ocorre o confronto contra o Bowser de Fogo, primeiro chefe do jogo. Diferentemente dos inimigos comuns, o chefe possui um padrão de ataque baseado no lançamento de projéteis, exigindo que o jogador combine movimentação, desvios e ataques para derrotá-lo.



Figura 6: Chefão da primeira fase

Já na segunda fase, é introduzida a Princesa de Gelo, que também realiza ataques por meio de projéteis, aumentando a dificuldade em relação aos inimigos encontrados anteriormente.



Figura 7: Inimigo da segunda fase

O chefe dessa fase é uma versão adaptada do personagem Mario, que utiliza o mesmo princípio de ataque à distância, encerrando a progressão do jogo em um confronto mais desafiador.



Figura 8: Chefão da segunda fase

Durante toda a execução da partida, são realizadas verificações contínuas de colisão entre personagem, inimigos e projéteis. Essas verificações determinam quando o jogador sofre dano, quando um inimigo é atingido e quando um projétil deve ser removido da tela. Caso o jogador perca todas as suas vidas durante os combates, é exibida a tela de derrota e um efeito sonoro é reproduzido.



Figura 9: Tela de derrota

4. Resultados Obtidos

Ao término do desenvolvimento, foi possível obter um jogo funcional. A aplicação desenvolvida reúne sistemas de movimentação, combate, renderização gráfica, interface com o usuário, gerenciamento de fases e reprodução de áudio, todos implementados integralmente em linguagem Assembly RISC-V utilizando o ambiente FPGRARS.

Durante os testes realizados, verificou-se o funcionamento correto da movimentação do personagem, incluindo deslocamento horizontal, salto, aplicação de gravidade e detecção de colisões com o cenário.

O sistema de câmera acompanhou adequadamente o deslocamento do jogador, permitindo a navegação por mapas maiores que a área visível na tela e garantindo uma experiência de jogo contínua.

As duas fases implementadas apresentaram ambientações distintas e elementos próprios, diferenciando-se tanto visualmente quanto pelos

inimigos encontrados e pela música de fundo. Na primeira fase, o jogador enfrenta plantas carnívoras, Bowser Esqueleto e o chefe Bowser de Fogo. Na segunda fase, ambientada em um cenário industrial, são introduzidos novos inimigos, como a Princesa de Gelo, culminando no confronto contra o chefe final Mario.

Essa divisão permitiu atender ao requisito de múltiplas áreas, proporcionando maior variedade durante a partida. O sistema de combate também apresentou funcionamento satisfatório, permitindo ao jogador alternar entre ataques à distância, realizados por meio de projéteis, e ataques corpo a corpo. As verificações de colisão entre personagem, inimigos e projéteis garantiram o correto funcionamento das mecânicas de combate, bem como do sistema de vidas implementado no jogo. Além das mecânicas principais, foram integrados recursos que contribuíram para a experiência do jogador, como interface gráfica (HUD), itens coletáveis, música de fundo e efeitos sonoros sincronizados com as ações realizadas durante a partida. Esses elementos aproximaram o projeto das características encontradas em jogos comerciais e tornaram a experiência mais dinâmica e imersiva.

5. Conclusões e Trabalhos futuros

O desenvolvimento deste projeto possibilitou aplicar, de forma prática, diversos conceitos estudados ao longo da disciplina de Organização e Arquitetura de Computadores, evidenciando os desafios envolvidos na implementação de aplicações gráficas utilizando linguagem Assembly RISC-V.

Durante o desenvolvimento foram explorados conceitos como manipulação de memória, renderização gráfica, tratamento de entrada do usuário, gerenciamento de sprites, detecção de colisões, organização modular do código e integração entre diferentes sistemas. O principal objetivo do projeto foi alcançado com a implementação de um jogo funcional inspirado em Mega Man 2, incorporando elementos do universo Super Mario Bros..

O resultado obtido foi um jogo composto por duas fases distintas, contendo diferentes inimigos, chefes exclusivos, sistema de combate com projéteis e ataques corpo a corpo, interface gráfica, itens coletáveis, efeitos sonoros e música de fundo, demonstrando a viabilidade do desenvolvimento de jogos completos em Assembly utilizando o ambiente

FPGRARS.

Além dos conhecimentos técnicos adquiridos, o projeto também proporcionou experiência em desenvolvimento colaborativo, exigindo a divisão de tarefas entre os integrantes do grupo e a integração de diferentes módulos em uma única aplicação. A organização do código e a modularização adotada durante a implementação contribuíram para facilitar essa integração e para reduzir dificuldades durante a etapa final do desenvolvimento.

Como trabalhos futuros, o jogo pode ser expandido com a implementação de novos tipos de inimigos e comportamentos mais complexos, inclusão de fases adicionais, ampliação do sistema de habilidades e desenvolvimento de novos chefes. Também podem ser realizadas otimizações na organização do código e na renderização dos elementos gráficos, visando facilitar futuras manutenções e a adição de novas funcionalidades.

Por fim, o projeto demonstrou que, mesmo utilizando uma linguagem de baixo nível como Assembly RISC-V, é possível desenvolver aplicações gráficas interativas e relativamente complexas, aplicando na prática os conhecimentos adquiridos ao longo da disciplina e consolidando conceitos fundamentais de arquitetura e organização de computadores.

6. Referências Bibliográficas

[1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface – RISC-V Edition. 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2020.

[2] D. A. Patterson and A. Waterman, The RISC-V Reader: An Open Architecture Atlas. Strawberry Canyon LLC, 2017.

[3] RARS Team, "RARS – RISC-V Assembler and Runtime Simulator." Disponível em: <https://github.com/TheThirdOne/rars>.

[4] Victor Lisboa, "FPGRARS." Disponível em: <https://github.com/victorlisboa/FPGRARS>.

[5] Victor Lisboa, "LAMAR – Laboratório de Arquitetura e Microarquitetura RISC-V." Disponível em: <https://github.com/victorlisboa/LAMAR>.

[6] Universidade de Brasília. Material didático da disciplina CIC0099 – Organização e Arquitetura de Computadores. Departamento de Ciência da Computação, 2026.

[7] Universidade de Brasília. Livrão de Organização e Arquitetura de Computadores. Material de apoio da

disciplina CIC0099, 2026.

[8] The Spriters Resource. "The Spriters Resource." Disponível em: <https://www.spriters-resource.com/>.

[9] Nintendo Co., Ltd. Super Mario Bros. Nintendo Entertainment System, 1985.

[10] Capcom Co., Ltd. Mega Man 2. Nintendo Entertainment System, 1988.